2 НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm

Рисунок 1 – Логотип среды разработки PyCharm

1) Python – это язык программирования, который широко используется в интернет-приложениях, разработке программного обеспечения, науке о данных и машинном обучении.

Достоинства языка:

- эффективный;
- прост в изучении;
- кроссплатформенный;
- совместим со всеми типами систем;
- Разработчики могут легко читать и понимать программы на Python, поскольку язык имеет базовый синтаксис, похожий на синтаксис английского;
- Большое количество доступных библиотек;
- повышает скорость разработки.

Недостатки языка:

- низкая производительность;
- динамическая типизация;
- потребление памяти.

Для данного проекта выбран язык программирования Python, так как он хорошо подходит для начинающих разработчиков и содержит большое количество подходящих библиотек.

PyCharm – это интегрированная среда разработки для Python от JetBrains.

Преимущества:

- умные возможности (автодополнение кода, рефакторинг);
- поддержка большого количества полезных библиотек;
- встроенный отладчик и консоль;
- поддержка веб-разработки;
- удобный интерфейс для Git;
- кроссплатформенность;
- большое количество плагинов для настройки среды под любые нужды.

Недостатки:

- может медленно работать с большими проектами на слабых компьютерах;
- множество функций могут перегружать интерфейс.

Эта IDE была выбрана, так как она хорошо подходит для разработки на Python и имеет удобный интерфейс.

Для создания графического интерфейса использовалась библиотека PyQt6. Её преимуществами являются:

- широкий набор готовых графических компонентов;
- возможность создания современного и визуально привлекательного интерфейса;
- поддержка объектно-ориентированного подхода;
- удобная система сигналов и слотов для обработки событий;
- кроссплатформенность – приложение может работать на Windows, Linux и macOS.

В рамках библиотеки PyQt6 были использованы следующие модули:

Модуль PyQt6.QtWidgets использовался для создания основных элементов интерфейса. Он содержит готовые компоненты, такие как кнопки, текстовые поля, метки, контейнеры и окна. Это значительно ускоряет разработку интерфейса и делает код более структурированным.

Модуль PyQt6.QtCore использовался для работы с базовой логикой приложения. Он предоставляет классы для работы со временем, таймерами, сигналами, слотами и различными служебными константами.

Модуль PyQt6.QtGui использовался для работы с графическим оформлением интерфейса. С его помощью реализованы цвета, эффекты оформления, а также управление текстовым курсором.

Также в программе использовались стандартные библиотеки Python.

Библиотека random применялась для генерации случайного тренировочного текста. Она позволяет выбирать случайные слова или предложения из заранее подготовленного словаря, что делает тренировку разнообразной.

Библиотека time использовалась для измерения времени тренировки. С её помощью рассчитывается длительность набора текста и скорость печати.

Библиотека sys использовалась для взаимодействия с системой и корректного завершения работы приложения.

Таким образом, выбранные библиотеки обеспечили удобство разработки, современный внешний вид программы, стабильную работу приложения и реализацию всех необходимых функций.

Скриншоты работы в выбранной среде приведены на рисунках ….

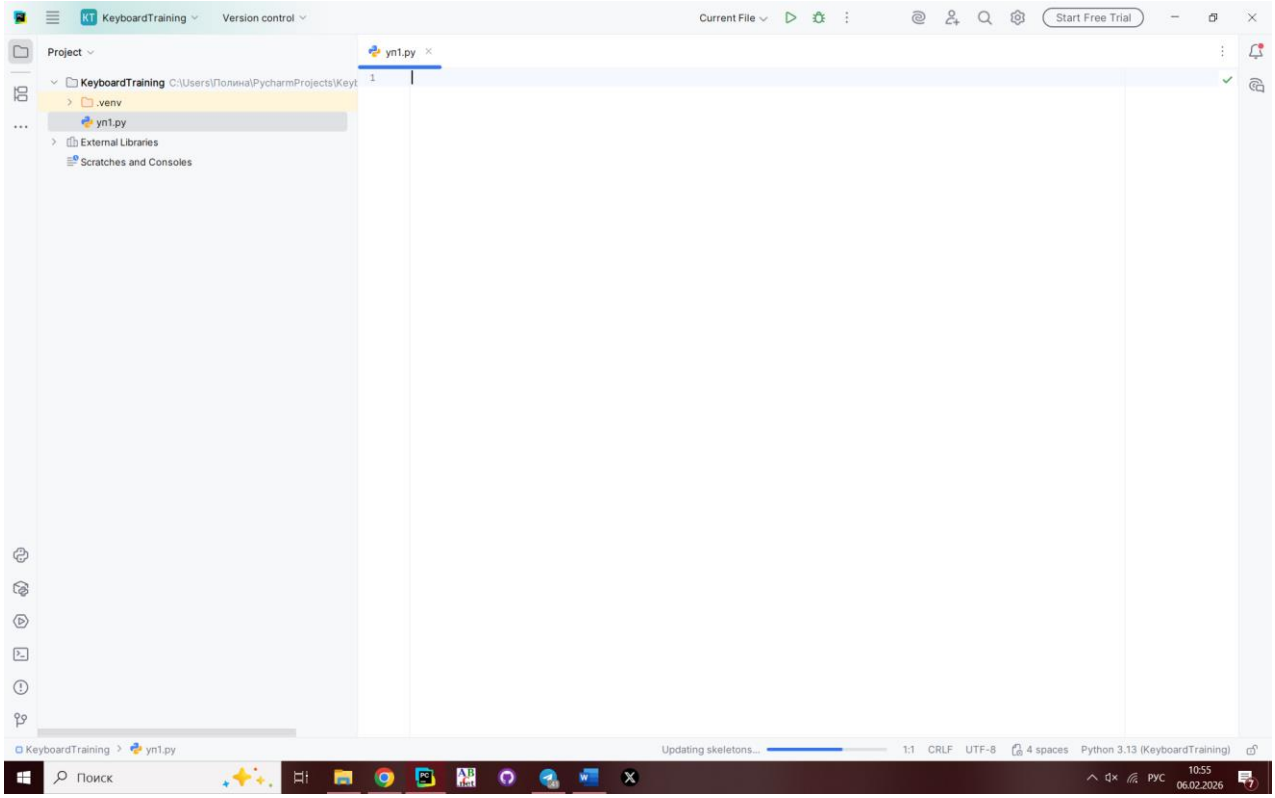

Рисунок – Интерфейс PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub.

Ссылка на профиль: https://github.com/glifflyn

Ссылка на созданный репозиторий: https://github.com/glifflyn/uchebnaya_practica_05.git

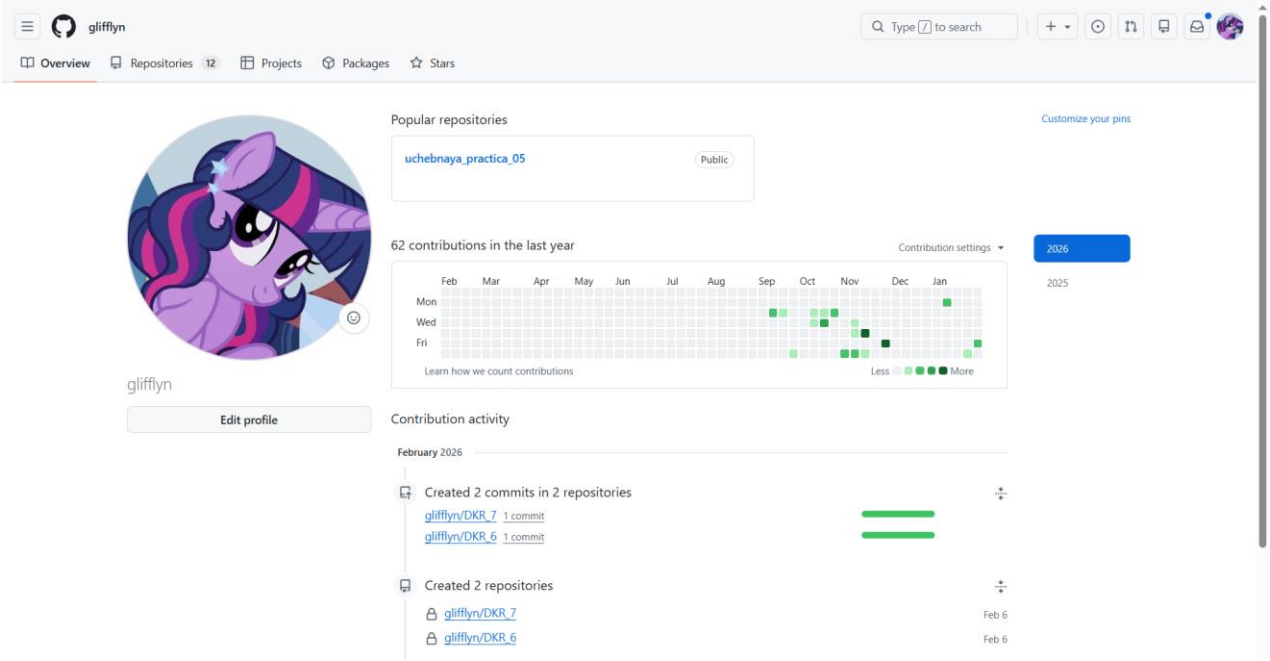

Рисунок – Профиль на GitHub

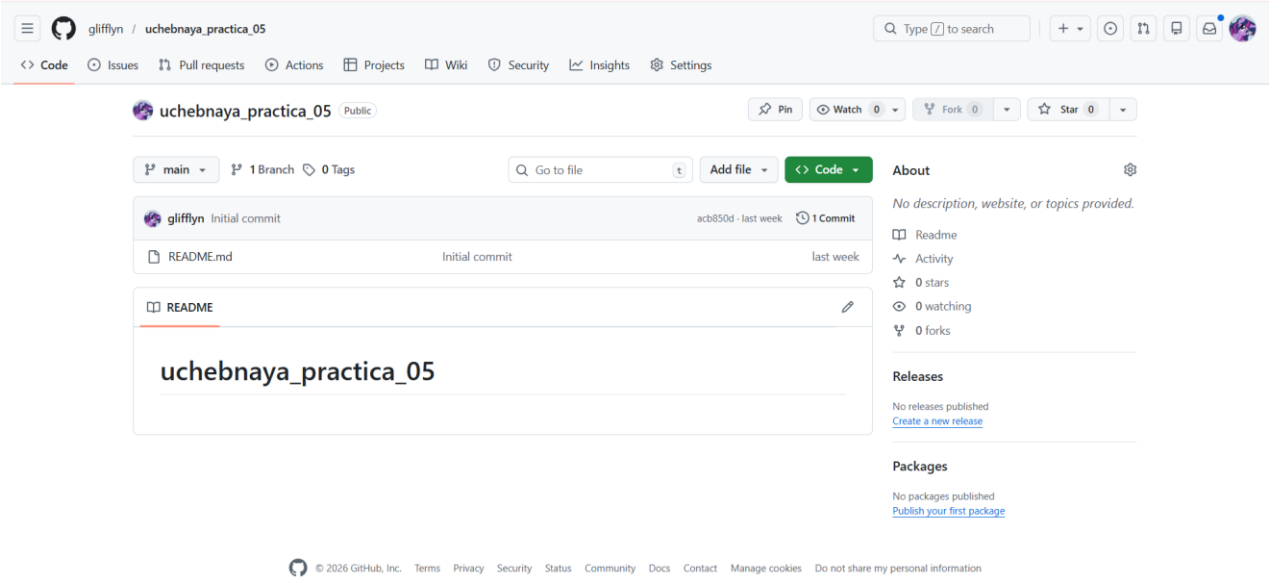

Рисунок – Созданный репозиторий

4 ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Клавиатура компьютера представляет собой периферийное устройство ввода, предназначенное для передачи символьной информации и управляющих команд в вычислительную систему. Конструктивно включает алфавитно-цифровой блок, функциональные клавиши, управляющие клавиши, блок навигации курсора и дополнительную цифровую панель (numpad).

Вводимые с клавиатуры символы в режиме реального времени отображаются на экране монитора в позиции курсора. Курсор – это графический индикатор текущей позиции ввода, определяющий место отображения следующего вводимого символа.

Наиболее распространённым стандартом раскладки является QWERTY, получившая наименование по последовательности символов в верхнем ряду алфавитно-цифрового блока [3].

Клавиатурный тренажёр – специализированное программное обеспечение, предназначенное для формирования навыков слепого десятипальцевого метода печати и повышения скорости текстового ввода. Десятипальцевый метод предполагает закрепление конкретного пальца за каждой клавишей и одновременное использование всех пальцев обеих рук. Такой подход обеспечивает повышение производительности ввода и снижение количества ошибок. Создатели этого метода не только поняли, как распределить пальцы по клавиатуре, но и разработали серию специальных упражнений. Они помогают мышцам запомнить место пальцев на клавиатуре. Подобным образом аппликатуре учат пианистов [1].

Программная реализация клавиатурного тренажёра, как правило, представляет собой учебную среду с функциональностью текстового редактора, включающую:

- отображение тренировочного текстового материала;
- автоматическую фиксацию ошибок;
- расчёт метрик набора (скорость печати, ритмичность, количество и типы опечаток и др.).

Методика обучения основана на формировании устойчивых моторных навыков (мышечной памяти) посредством многократного повторения букв, буквосочетаний, слов и предложений. В результате снижается когнитивная нагрузка на пользователя, так как процесс нажатия клавиш автоматизируется и осуществляется без постоянного визуального контроля.

Дополнительно формируется ритмичность набора – равномерность и согласованность движений пальцев. Это способствует повышению общей эффективности ввода и уменьшению утомляемости. При достижении устойчивого навыка пользователь способен осуществлять быстрый и точный ввод текста без визуального контроля клавиатуры.

Современные клавиатурные тренажёры часто реализуются в формате онлайн-приложений, что обеспечивает доступность, удобство использования и возможность хранения статистики прогресса пользователя.

Эффективный программный продукт данного класса должен учитывать индивидуальные особенности обучающегося, включая текущий уровень подготовки, скорость освоения материала и характер допускаемых ошибок.

Регулярные кратковременные занятия (10–15 минут в день) способствуют устойчивому увеличению скорости печати и снижению количества опечаток [2].

Таким образом, клавиатурный тренажёр представляет собой программно-методический комплекс, направленный на автоматизацию навыков текстового ввода, развитие моторной памяти и повышение общей эффективности работы пользователя с компьютерной клавиатурой.

4.1.1 Анализ проблемы

В условиях цифровизации и широкого использования компьютерных технологий навыки быстрого и точного клавиатурного ввода становятся неотъемлемой частью профессиональной деятельности и повседневной жизни. Несмотря на это, значительная часть пользователей продолжает набирать текст медленно, с частыми ошибками и постоянным визуальным контролем клавиатуры, что снижает общую продуктивность работы и увеличивает утомляемость.

Существующие клавиатурные тренажёры не всегда в полной мере решают данную проблему. Многие из них обладают ограниченным функционалом, устаревшим интерфейсом или не учитывают типичные ошибки. Кроме того, часть приложений ориентирована исключительно на механическое повторение упражнений без достаточной обратной связи и мотивационных элементов, что снижает интерес к регулярным тренировкам.

Актуальной проблемой также является недостаточная адаптивность обучающих программ. Большинство тренажёров используют стандартные наборы упражнений и фиксированную сложность, что не позволяет эффективно выстраивать персонализированную траекторию обучения. В результате начинающим пользователям задания могут показаться слишком сложными, а более опытным – однообразными и неэффективными.

Таким образом, возникает необходимость в разработке современного клавиатурного тренажёра, который сочетал бы удобный интерфейс, адаптивную систему обучения и анализ ошибок пользователя. Решение данной проблемы позволит повысить эффективность освоения навыков слепого десятипальцевого набора и расширить круг пользователей, заинтересованных в регулярном обучении.

В современном обществе работа с текстовой информацией занимает значительную часть учебной и профессиональной деятельности. При этом эффективность взаимодействия с компьютером во многом зависит от уровня владения клавиатурным вводом. Низкая скорость печати и большое

количество ошибок приводят к потере времени, снижению концентрации внимания и росту утомляемости пользователя.

Анализ существующих программ клавиатурного обучения показывает, что многие из них не соответствуют требованиям современных пользователей. Часть тренажёров предлагает однотипные упражнения без учёта прогресса обучающегося, что делает процесс обучения монотонным и снижает мотивацию. Кроме того, некоторые приложения не предоставляют детализированную статистику результатов, что затрудняет оценку эффективности тренировок и выявление слабых сторон пользователя.

Отдельной проблемой является отсутствие гибкой настройки сложности и адаптации заданий под индивидуальные особенности обучаемого. Пользователи с разным уровнем подготовки вынуждены проходить одинаковые этапы обучения, что негативно сказывается на темпе освоения навыков. Также нередко не учитываются современные требования к интерфейсу и пользовательскому опыту, такие как наглядность, удобство навигации и визуальная обратная связь.

В связи с вышеизложенным становится очевидной необходимость разработки приложения клавиатурного тренажёра, ориентированного на персонализацию обучения, анализ действий пользователя и повышение мотивации. Реализация такого приложения позволит создать более эффективную и комфортную среду для формирования навыков быстрого и точного клавиатурного ввода.

4.1.2 Обзор аналогов

Приведем примеры нескольких клавитаутрных тренажеров. Для этого требуется рассмотреть преимущества и недостатки каждого аналога и выделить их общие черты.

Первый аналог – это клавиатурный тренажер RomanPro.ru. Интерфейс представлен на рисунке .

Преимущества:

- выбор режимов;
- отображение длины текста, допущенных ошибок, процента выполнения и средней скорости печати.

Недостатки:

- не показано как должны располагаться пальцы;
- обязателен доступ к сети;
- доступно обучение только для русской раскладки.

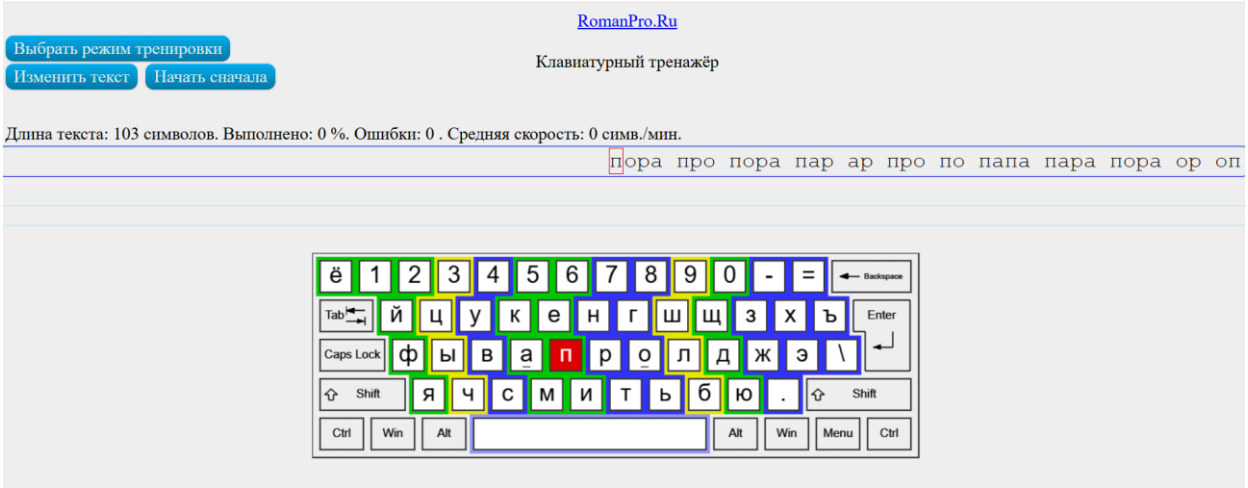

Рисунок – Интерфейс RomanPro.ru

Следующий аналог – клавиатурный тренажер blindtyping.com. Показан на рисунке .

Преимущества:

- доступно множество языков;
- доступно несколько режимов;
- отображение точности и скорости печати.

Недостатки:

- отображение двуигающегося силуэта рук перед клавиатурой может мешать процессу;
- требуется доступ к сети;

– большое количество кнопок на экране, которые могут путать новых пользователей.

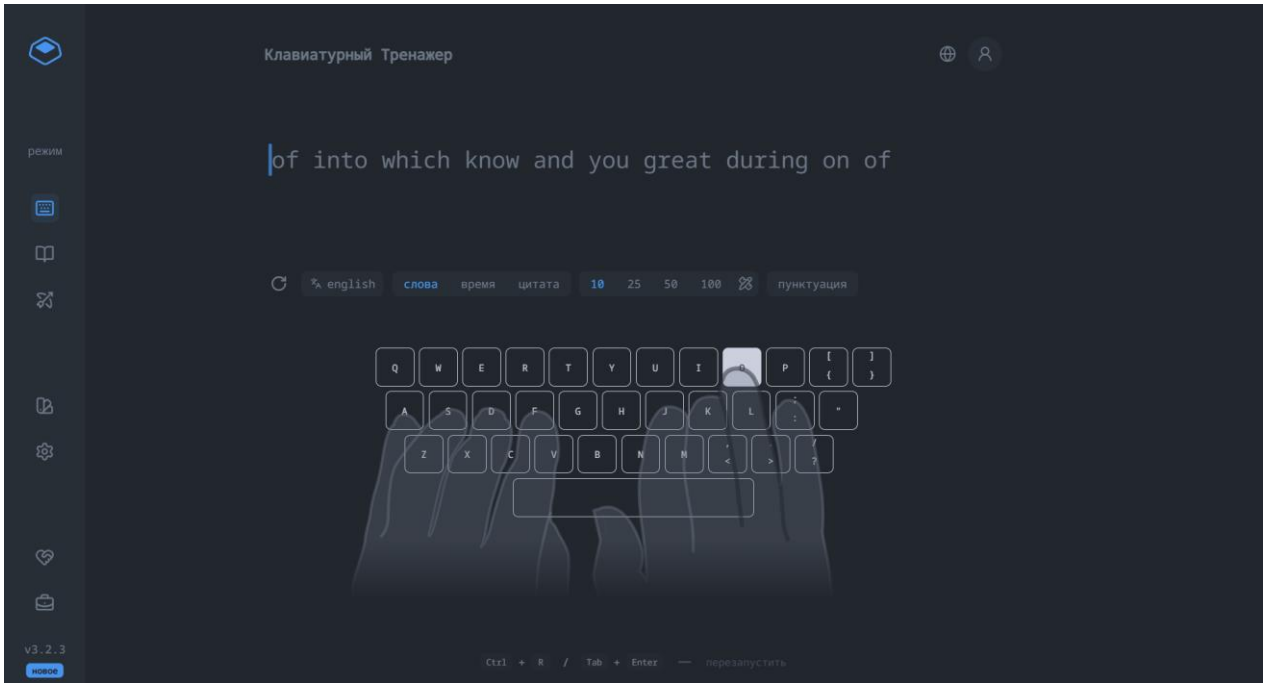

Рисунок – Интерфейс blindtyping.com

Третий рассмотренный аналог – это клавиатурный тренажер Stamina-online.com. Показан на рисунке .

Преимущества:

– постоянное отображение положений пальцев с помощью привязки цвета;
– отображение скорости и точности печати.

Недостатки:

– обязателен доступ к сети;
– нет вывода об ошибках во время печати, что может замедлить процесс.

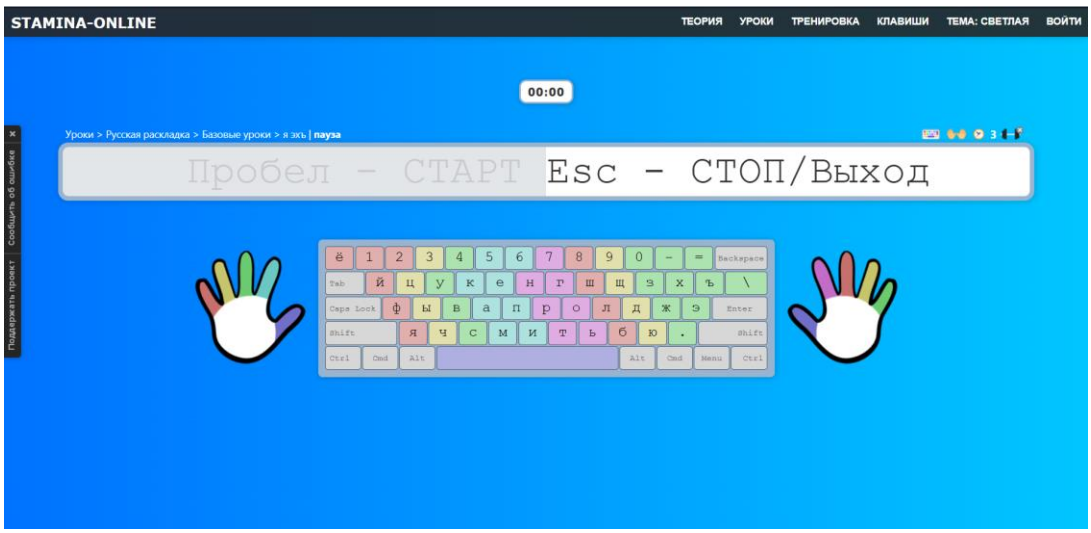

Рисунок – Интерфейс Stamina-online.com

Последний аналог – это клавиатурынй тренажер Staminaon.com. Показан на рисунке .

Преимущества:

- есть выбор раскладки;
- вывод скорости печати и количества ошибок.

Недостатки:

- отвлекающий фон;
- нет обозначений для расстановки пальцев;
- на основной странице слишком примитивный интерфейс с большим количеством текста;
- требуется доступ к сети;
- нет вывода об ошибках во время печати, что может замедлить процесс.

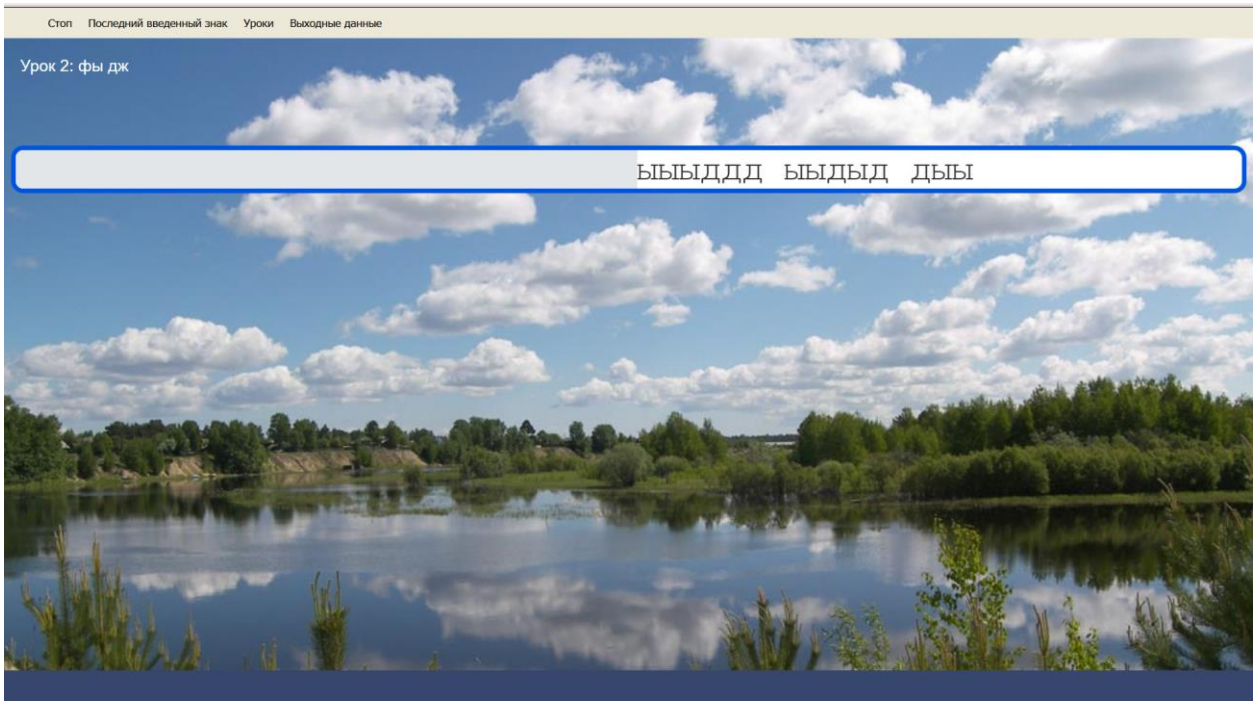

Рисунок – Интерфейс Staminaon.com

Можно сделать вывод, что для всех рассмотренных аналогов требуется постоянное подключение к сети, и чаще всего нет удобного отбражения ошибок во время печати или понятного объяснения расстановки пальцев. В преимуществах все аналоги схожи в отображении скорости и точности печати, что является основными показателями для клавиатурного тренажера.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

Алгоритм работы клавиатурного тренажера представлен на рисунке 1.

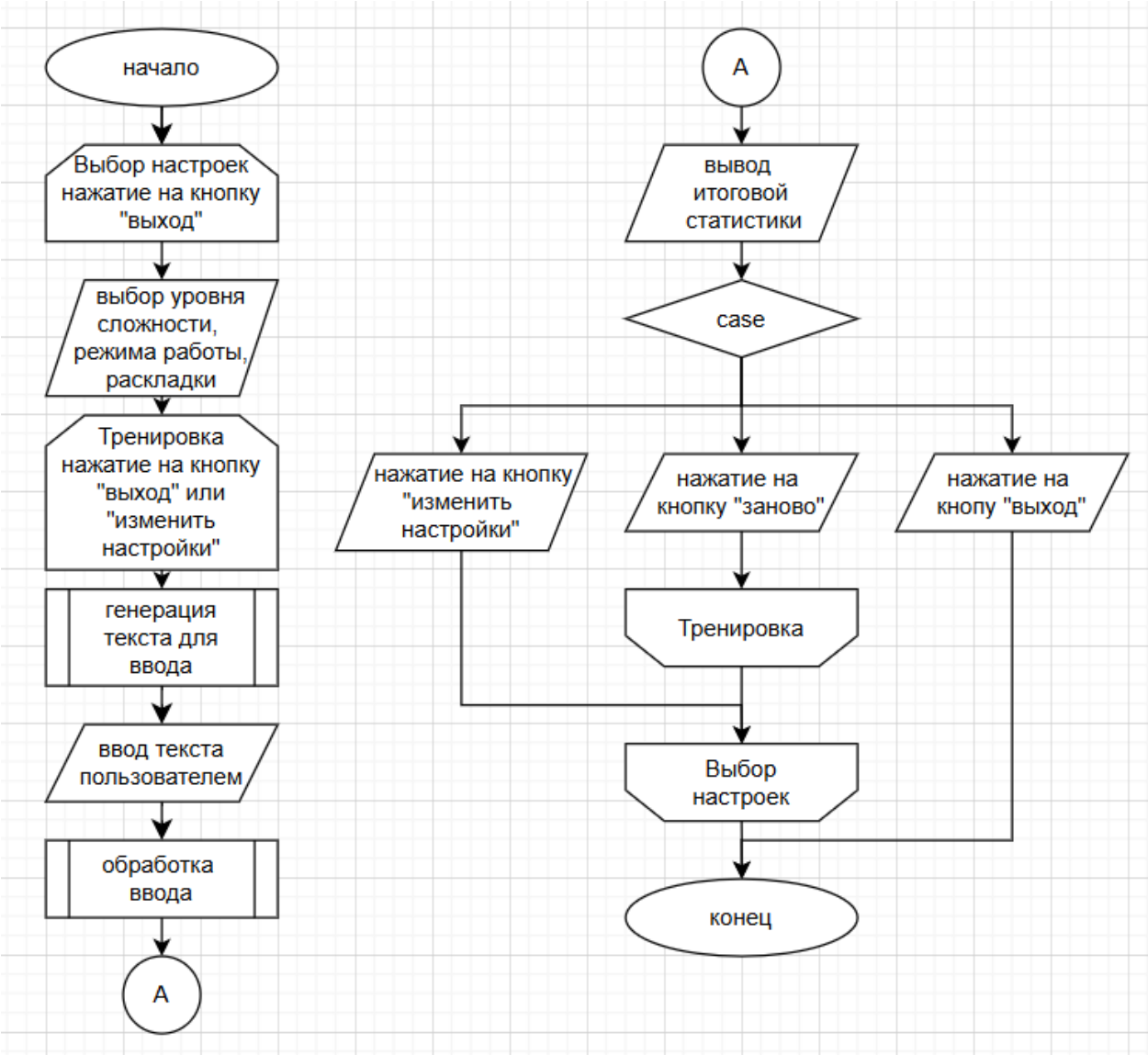

Рисунок 1 – Алгоритм работы клавиатурного тренажера

Алгоритм представляет собой процесс выбора желаемых настроек, функцию генерации текста для ввода и сам процесс ввода текста

пользователем. Он также включает в себя функцию обработки ввода, которая нужна для вывода итоговой статистики.

Алгоритм работы функции генерации текста представлен на рисунке 2.

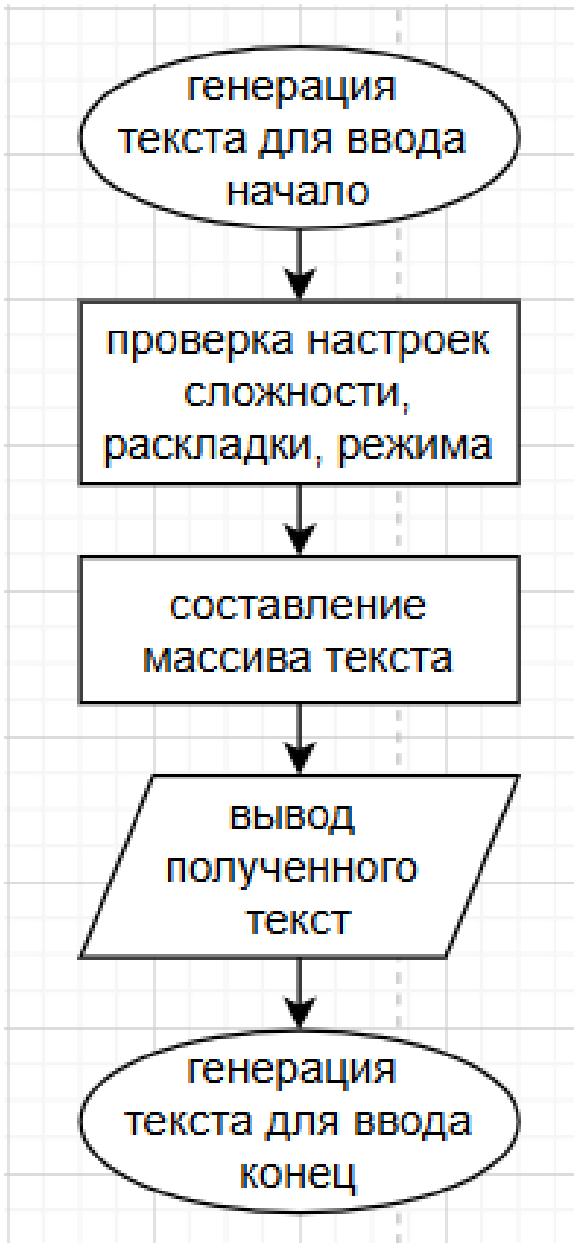

Рисунок 2 – Алгоритм работы функции для генерации текста

Эта функция генерирует массив текста на основе выбранных пользователем настроек.

Алгоритм работы функции для обработки ввода пользователя представлен на рисунке 3.

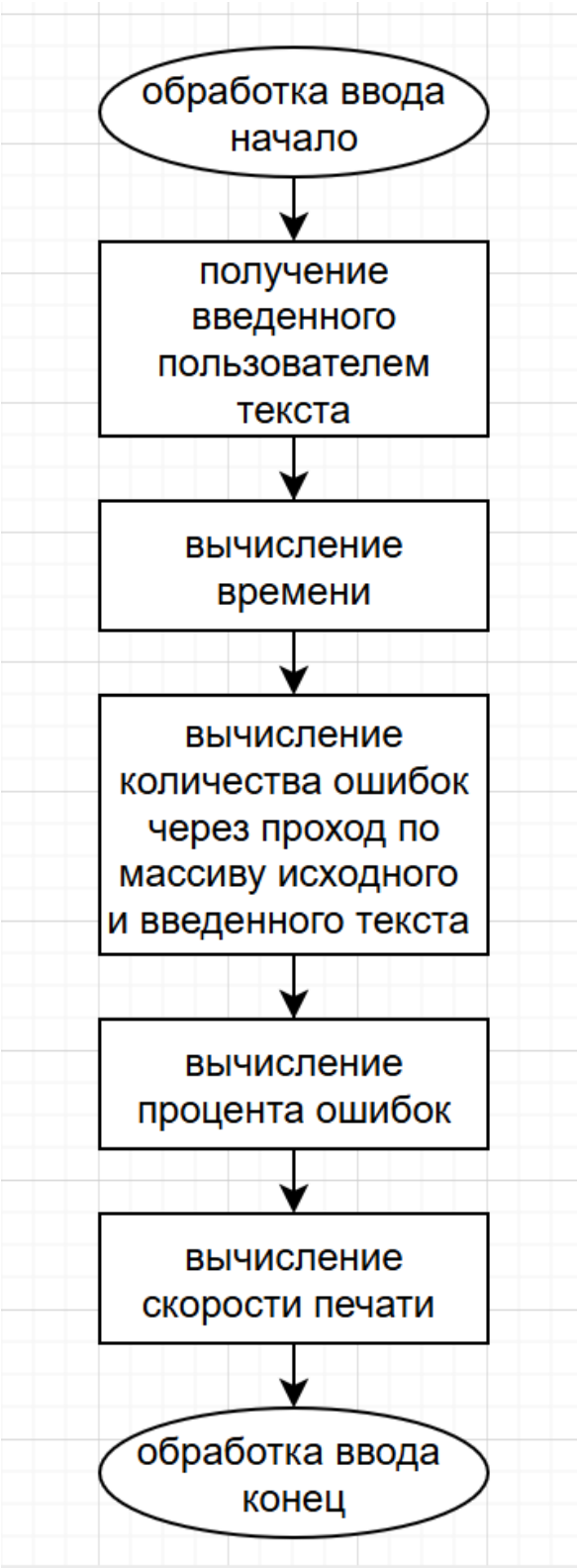

Рисунок 3 – Алгоритм работы обработки ввода пользователя

Эта функция отвечает за анализ введенных символов через сравнение с требуемым текстом и вычисление параметров качества набора.

4.2.3 Проектирование интерфейса

На рисунках 4-9 приводятся прототипы интерфейса, нарисованные в Figma.

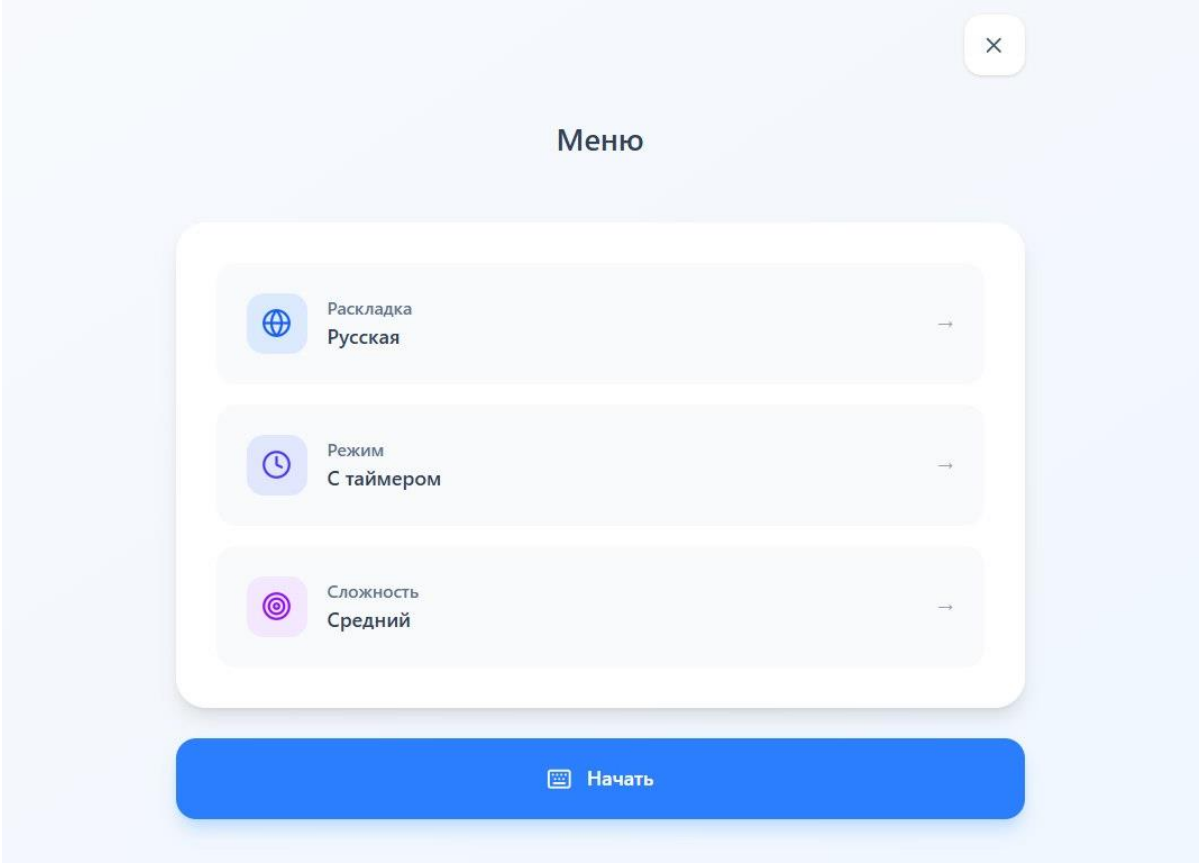

Рисунок 4 – Прототип интерфейса главного меню

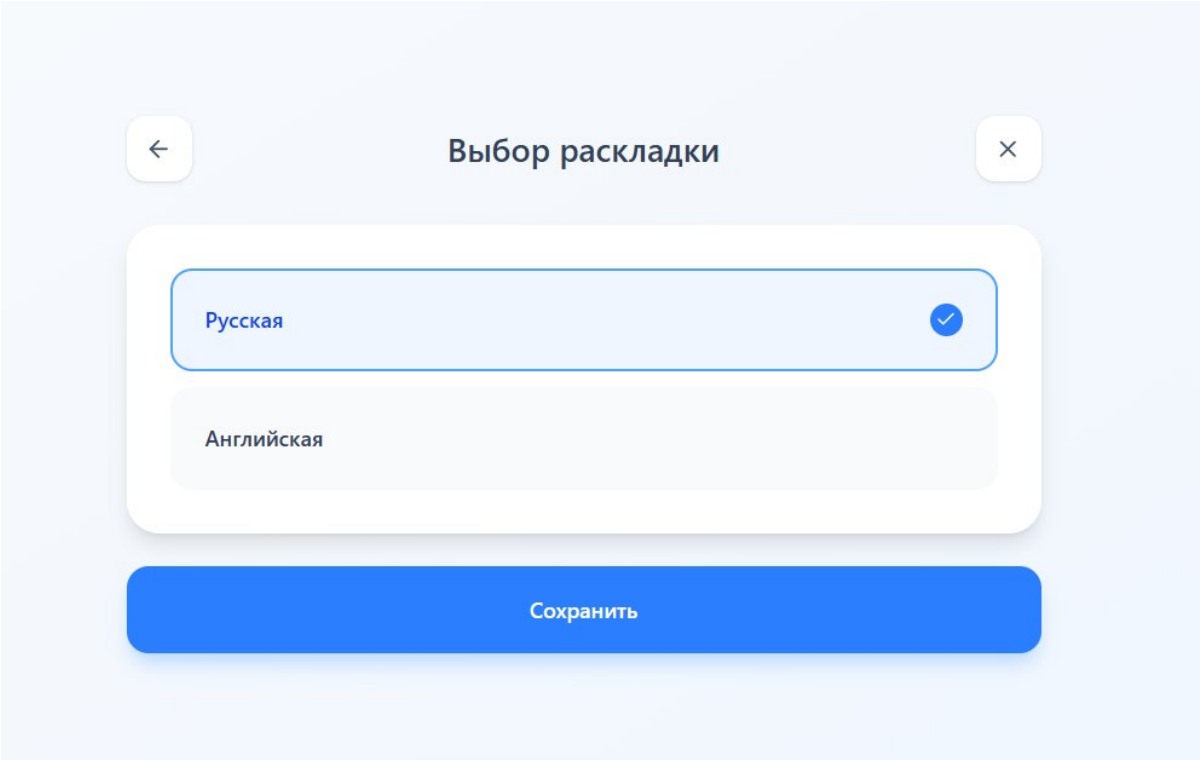

Рисунок 5 – Прототип интерфейса экрана изменения раскладки

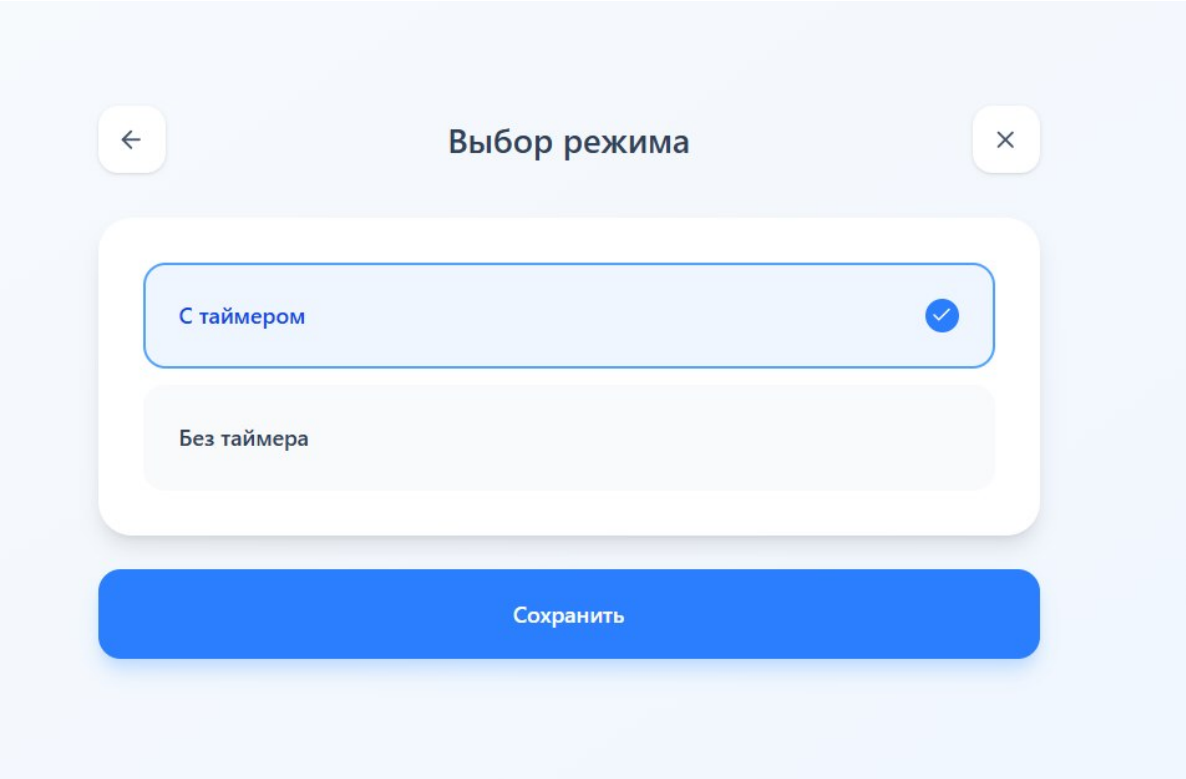

Рисунок 6 – Прототип интерфейса экрана изменения режима

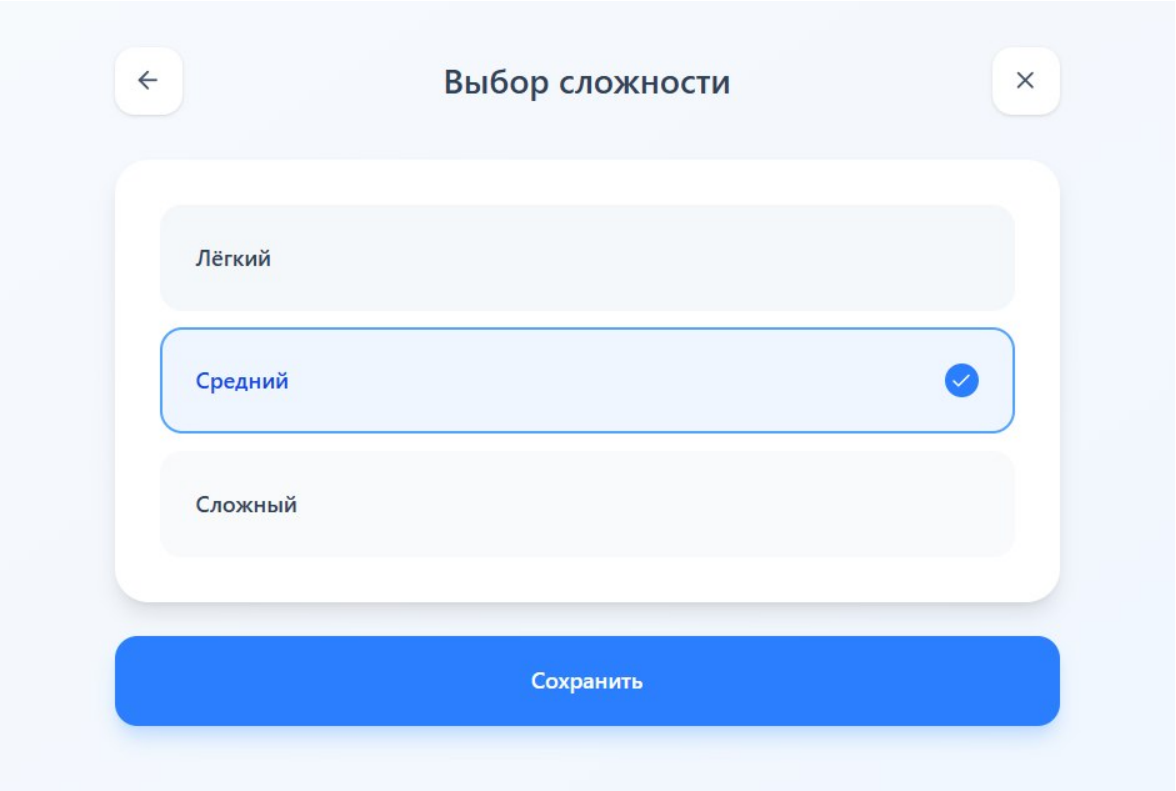

Рисунок 7 – Прототип интерфейса экрана изменения уровня сложности

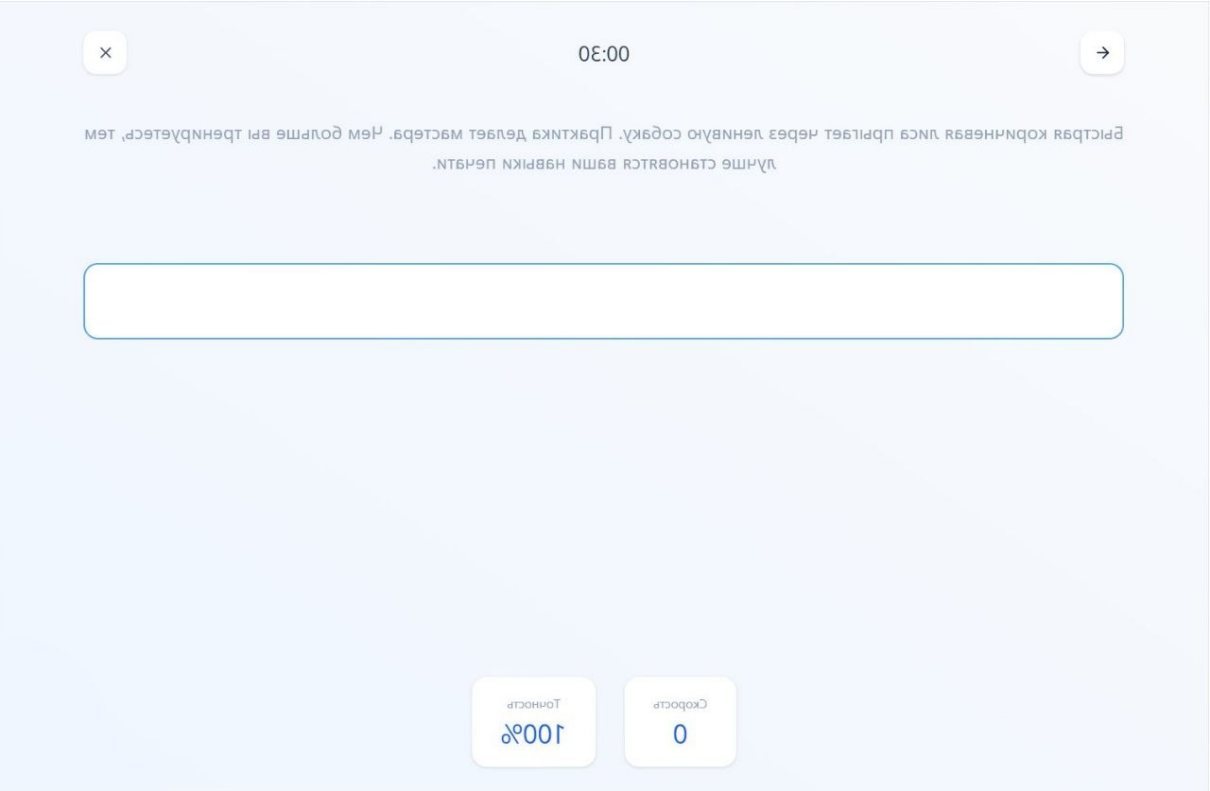

Рисунок 8 – Прототип интерфейса основного экрана тренировки

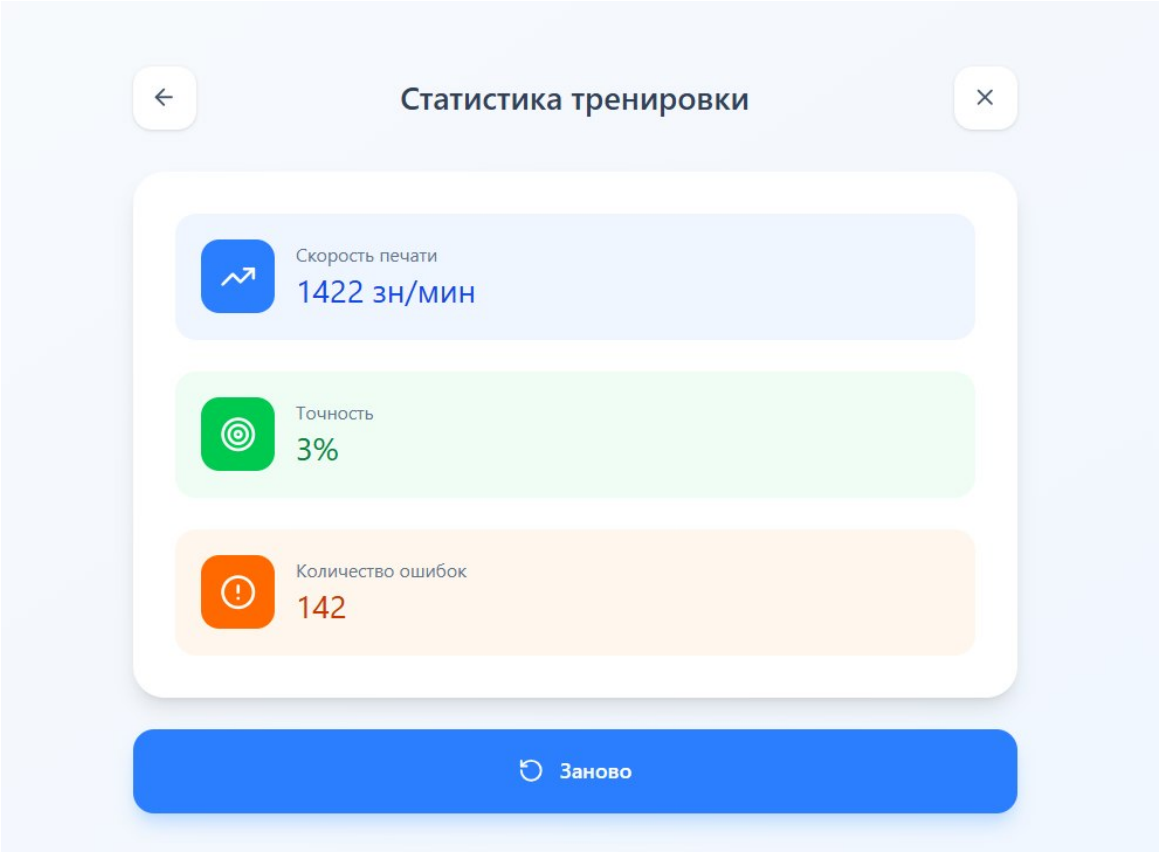

Рисунок 9 – Прототип интерфейса экрана статистики

В данном разделе рассмотрены алгоритмы работы системы. Рассмотрен основной алгоритм программы, а также алгоритмы двух ключевых функций для генерации текста и обработки ввода. Приводится прототипы интерфейса, созданные в Figma.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования программы «Клавиатурный тренажер», разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано в среде PyCharm на языке Python.

Программное обеспечение состоит из шести классов:

– MenuScreen – начальное окно меню

– SelectionScreen – универсальные окно выбора настроек языка, раскладки, режима

– TrainingScreen – основное окно тренировки

– StatsScreen – окно статистики

– TrainerApp – хранит глоабльное состояние приложения и управляет навигацией

– ShadowFrame

Функция generate_practice_text отвечает за генерацию тренировочного текста в зависимости от выбранного языка, уровня сложности и режима тренировки. На основе словаря VOCABULARY функция случайным образом формирует строку из слов или предложений. Исходный код программного обеспечения представлен в приложении А. Исходный код функции представлен на рисунке 1.

```
71  def generate_practice_text(lang, diff, is_timer):  1 usage
72      pool = VOCABULARY[lang][diff]
73      if diff == "Лёгкий":
74          count = 30 if is_timer else 15
75      else:
76          count = 15 if is_timer else 4
77      return " ".join(random.choices(pool, k=count))
```

Рисунок 1 – Исходный код функции generate_practice_text

Класс ShadowFrame наследуется от класса QFrame и предназначен для создания карточек интерфейса с эффектом тени. В конструкторе класса задаётся стиль карточки, цвет тени, степень размытия и смещение. Исходный код класса представлен на рисунке 2.

```
80  class ShadowFrame(QFrame):  5 usages
81      def __init__(self):
82          super().__init__()
83          self.setProperty( name: "class",  value: "Card")
84          shadow = QGraphicsDropShadowEffect()
85          shadow.setBlurRadius(20)
86          shadow.setColor(QColor( *__args: 0, 0, 0, 20))
87          shadow.setOffset( *__args: 0, 5)
88          self.setGraphicsEffect(shadow)
```

Рисунок 2 – Исходный код класса ShadowFrame

Класс SelectionScreen отвечает за создание экранов выбора параметров тренировки. С его помощью реализован выбор раскладки, режима работы и сложности.

Функция __init__ создаёт интерфейс экрана, включая кнопки возврата, закрытия, список вариантов выбора и кнопку сохранения. Исходный код функции представлен на рисунках 3 и 4.

```
92  class SelectionScreen(QWidget):  3 usages
93      def __init__(self, parent, title, options, setting_key):
94          super().__init__(parent)
95          self.parent_app = parent
96          self.key, self.val = setting_key, parent.settings[setting_key]
97          layout = QVBoxLayout(self)
98          layout.setContentsMargins( *__args: 30, 30, 30, 30)
99
100         top_bar = QHBoxLayout()
101         back = QPushButton("←")
102         back.setFixedSize(45, 45)
103         back.setProperty( name: "class",  value: "NavButton")
104         back.clicked.connect(lambda: self.parent_app.stack.setCurrentIndex(0))
105         close = QPushButton("×")
106         close.setFixedSize(45, 45)
107         close.setProperty( name: "class",  value: "CloseButton")
108         close.clicked.connect(self.window().close)
109         t_lbl = QLabel(title)
110         t_lbl.setStyleSheet("font-size: 30px; font-weight: bold;")
111         top_bar.addWidget(back)
112         top_bar.addStretch()
113         top_bar.addWidget(t_lbl)
114         top_bar.addStretch()
115         top_bar.addWidget(close)
116         layout.addLayout(top_bar)
```

Рисунок 3 – Фрагмент исходного кода функции __init__ класса SelectionScreen

```
118        layout.addStretch()
119        container = QWidget()
120        container.setFixedWidth(500)
121        c_lay = QVBoxLayout(container)
122        self.card = ShadowFrame()
123        card_lay = QVBoxLayout(self.card)
124        self.btns = []
125        for o in options:
126            btn = QPushButton(o)
127            btn.setProperty( name: "class",  value: "OptionButton")
128            btn.setProperty( name: "selected", str(o == self.val).lower())
129            btn.clicked.connect(lambda ch, v=o: self.select(v))
130            card_lay.addWidget(btn)
131            self.btns.append(btn)
132        c_lay.addWidget(self.card)
133        save = QPushButton("Сохранить")
134        save.setProperty( name: "class",  value: "PrimaryButton")
135        save.clicked.connect(self.save_data)
136        c_lay.addWidget(save)
137        layout.addWidget(container, alignment=Qt.AlignmentFlag.AlignCenter)
138        layout.addStretch()
```

Рисунок 4 – Фрагмент исходного кода функции __init__ класса SelectionScreen

Функция select изменяет выбранный пользователем параметр и обновляет стиль кнопок. Исходный код функции представлен на рисунке 5.

```
140    def select(self, v):  1 usage
141        self.val = v
142        for b in self.btns:
143            b.setProperty("selected", str(b.text() == v).lower())
144            b.style().unpolish(b)
145            b.style().polish(b)
146
```

Рисунок 5 – Исходный код функции select

Функция save_data сохраняет выбранное значение в настройках приложения и обновляет главное меню. Исходный код функции представлен на рисунке 6.

```
147        def save_data(self):  1 usage
148            self.parent_app.settings[self.key] = self.val
149            self.parent_app.menu_scr.refresh_ui()
150            self.parent_app.stack.setCurrentIndex(0)
151
```

Рисунок 6 – Исходный код функции save_data

Класс MenuScreen реализует главное меню приложения.

Функция __init__ создаёт интерфейс главного меню, содержащий строки выбора параметров и кнопку запуска тренировки. Исходный код функции представлен на рисунках 7 и 8.

```
154    class MenuScreen(QWidget):  1 usage
155        def __init__(self, parent):
156            super().__init__(parent)
157            self.parent_app = parent
158            layout = QVBoxLayout(self)
159            layout.setContentsMargins( *__args: 30, 30, 30, 30)
160
161            top_bar = QHBoxLayout()
162            top_bar.addStretch()
163            close = QPushButton("×")
164            close.setFixedSize(45, 45)
165            close.setProperty( name: "class",  value: "CloseButton")
166            close.clicked.connect(self.window().close)
167            top_bar.addWidget(close)
168            layout.addLayout(top_bar)
169
170            layout.addStretch()
171            container = QWidget()
172            container.setFixedWidth(500)
173            c_lay = QVBoxLayout(container)
174            title = QLabel("Меню")
175            title.setStyleSheet("font-size: 30px; font-weight: bold;")
176            title.setAlignment(Qt.AlignmentFlag.AlignCenter)
177            c_lay.addWidget(title)
178            c_lay.addSpacing(25)
```

Рисунок 7 – Фрагмент исходного кода функции __init__ класса MenuScreen

```
180        card = ShadowFrame()
181        card_lay = QVBoxLayout(card)
182        self.labels = {}
183        for key, idx in [("Раскладка", 2), ("Режим", 3), ("Сложность", 4)]:
184            btn = QPushButton()
185            btn.setProperty( name: "class",  value: "MenuRow")
186            btn.setFixedHeight(90)
187            blay = QVBoxLayout(btn)
188            t = QLabel(key)
189            t.setStyleSheet("color: #64748b; font-size: 20px; background: transparent;")
190            v = QLabel(parent.settings[key])
191            v.setStyleSheet("font-weight: bold; font-size: 24px; background: transparent;")
192            blay.addWidget(t)
193            blay.addWidget(v)
194            btn.clicked.connect(lambda ch, i=idx: self.parent_app.stack.setCurrentIndex(i))
195            card_lay.addWidget(btn)
196            self.labels[key] = v
197
198        c_lay.addWidget(card)
199        c_lay.addSpacing(25)
200        start = QPushButton("Начать")
201        start.setProperty( name: "class",  value: "PrimaryButton")
202        start.clicked.connect(self.start_training)
203        c_lay.addWidget(start)
204        layout.addWidget(container, alignment=Qt.AlignmentFlag.AlignCenter)
205        layout.addStretch()
```

Рисунок 8 – Фрагмент исходного кода функции __init__ класса MenuScreen

Функция refresh_ui обновляет отображаемые параметры после изменения настроек. Исходный код функции представлен на рисунке 9.

```
207    def refresh_ui(self):  1 usage (1 dynamic)
208        for k, v in self.parent_app.settings.items():
209            if k in self.labels: self.labels[k].setText(v)
```

Рисунок 9 – Исходный код функции refresh_ui

Функция start_training запускает тренировочный режим, инициализируя новый сеанс. Исходный код функции представлен на рисунке 10.

```
211        def start_training(self):  1 usage
212            self.parent_app.train_scr.init_game()
213            self.parent_app.stack.setCurrentIndex(1)
```

Рисунок 10 – Исходный код функции start_training

Класс TrainingScreen отвечает за реализацию основного процесса тренировки печати.

Функция __init__ создаёт интерфейс тренировки: строку таймера, поле отображения текста, поле ввода, а также блок статистики. Исходный код функции представлен на рисунке 11 и 12.

```
217    class TrainingScreen(QWidget):  1 usage
218        def __init__(self, parent):
219            super().__init__(parent)
220            self.parent_app = parent
221            self.timer = QTimer()
222            self.timer.timeout.connect(self.tick)
223            self.last_len = 0
224
225            layout = QVBoxLayout(self)
226            layout.setContentsMargins( *__args: 40, 40, 40, 40)
227
228            top_bar = QHBoxLayout()
229            back = QPushButton("←")
230            back.setFixedSize(45, 45)
231            back.setProperty( name: "class",  value: "NavButton")
232            back.clicked.connect(self.go_back)
233            close = QPushButton("×")
234            close.setFixedSize(45, 45)
235            close.setProperty( name: "class",  value: "CloseButton")
236            close.clicked.connect(self.window().close)
237            self.lbl_timer = QLabel("00:00")
238            self.lbl_timer.setStyleSheet("font-size: 30px; font-weight: bold; color: #1e293b;")
239            top_bar.addWidget(back)
240            top_bar.addStretch()
241            top_bar.addWidget(self.lbl_timer)
242            top_bar.addStretch()
243            top_bar.addWidget(close)
244            layout.addLayout(top_bar)
245
246            font_style = "font-family: 'Consolas', 'Courier New', monospace; font-size: 30px; padding: 16px;"
247
```

Рисунок 11 – Фрагмент исходного кода функции __init__ класса TrainingScreen

```
248        self.ref_container = ShadowFrame()
249        self.ref_lay = QVBoxLayout(self.ref_container)
250        self.ref_lay.setContentsMargins( *__args: 0, 0, 0, 0)
251        self.ref = QTextEdit()
252        self.ref.setReadOnly(True)
253        self.ref.setLineWrapMode(QTextEdit.LineWrapMode.NoWrap)
254        self.ref.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
255        self.ref.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
256        self.ref.setStyleSheet(font_style + " color: #94a3b8; background: transparent; border: none;")
257        self.ref.setFixedHeight(80)
258        self.ref_lay.addWidget(self.ref)
259        layout.addWidget(self.ref_container)
260
261        self.inp = QTextEdit()
262        self.inp.setLineWrapMode(QTextEdit.LineWrapMode.NoWrap)
263        self.inp.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
264        self.inp.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
265        self.inp.setStyleSheet(font_style + " border: 2px solid #3b82f6; border-radius: 12px; background: white;")
266        self.inp.setFixedHeight(70)
267        self.inp.textChanged.connect(self.on_change)
268
269        self.inp.horizontalScrollBar().valueChanged.connect(self.ref.horizontalScrollBar().setValue)
270
271        layout.addWidget(self.inp)
272        layout.addSpacing(25)
273
274        s_lay = QHBoxLayout()
275        self.s_speed = self.stat_mini( t: "Скорость",  v: "0")
276        self.s_acc = self.stat_mini( t: "Точность",  v: "100%")
277        s_lay.addWidget(self.s_speed)
278        s_lay.addWidget(self.s_acc)
279        layout.addLayout(s_lay)
280        layout.addStretch()
```

Рисунок 12 – Фрагмент исходного кода функции __init__ класса TrainingScreen

Функция stat_mini создаёт карточки мини-статистики. Исходный код функции представлен на рисунке 13.

```
282    def stat_mini(self, t, v):  2 usages
283        f = ShadowFrame()
284        f.setFixedSize(200, 100)
285        l = QVBoxLayout(f)
286        l.setAlignment(Qt.AlignmentFlag.AlignCenter)
287        lbl_t = QLabel(t)
288        lbl_t.setStyleSheet("color: #64748b; font-size: 16px;")
289        lbl_v = QLabel(v)
290        lbl_v.setStyleSheet("font-size: 25px; font-weight: bold; color: #3b82f6;")
291        l.addWidget(lbl_t)
292        l.addWidget(lbl_v)
293        f.val = lbl_v
294        return f
```

Рисунок 13 – Исходный код функции stat_mini

Функция init_game запускает новую игру, генерирует тренировочный текст, сбрасывает статистику и запускает таймер. Исходный код функции представлен на рисунке 14.

```
296     def init_game(self):  2 usages (2 dynamic)
297         s = self.parent_app.settings
298         is_timer = s["Режим"] == "С таймером"
299         self.target = generate_practice_text(s["Раскладка"], s["Сложность"], is_timer)
300
301         self.errs, self.active, self.start_t = 0, False, None
302         self.last_len = 0
303
304         self.inp.clear()
305         self.inp.setReadOnly(False)
306
307         if is_timer:
308             timer_mapping = {"Лёгкий": 15, "Средний": 30, "Сложный": 45}
309             self.time_limit = timer_mapping[s["Сложность"]]
310             m, sec = divmod(self.time_limit, 60)
311             self.lbl_timer.setText(f"{m:02d}:{sec:02d}")
312             self.lbl_timer.show()
313         else:
314             self.time_limit = 0
315             self.lbl_timer.hide()
316
317         self.on_change()
318
319         self.s_speed.val.setText("0")
320         self.s_acc.val.setText("100%")
321         self.inp.setFocus()
```

Рисунок 14 – Исходный код функции init_game

Функция go_back останавливает тренировку и возвращает пользователя в меню. Исходный код функции представлен на рисунке 15.

```
323     def go_back(self):  1 usage
324         self.timer.stop()
325         self.parent_app.stack.setCurrentIndex(0)
326
```

Рисунок 15 – Исходный код функции go_back

Функция on_change отслеживает ввод текста, сравнивает введённые символы с эталоном, подсвечивает ошибки и корректные символы, а также завершает тренировку при окончании текста. Исходный код функции представлен на рисунках 16 и 17.

```python
327     def on_change(self):  2 usages
328         t = self.inp.toPlainText()
329 
330         if '\n' in t:
331             t = t.replace('\n', '')
332             self.inp.setPlainText(t)
333             cur = self.inp.textCursor()
334             cur.movePosition(QTextCursor.MoveOperation.End)
335             self.inp.setTextCursor(cur)
336             return
337 
338         if not self.active and len(t) > 0:
339             self.active = True
340             self.start_t = time.time()
341             self.timer.start(500)
342 
343         if len(t) > 0 and len(t) <= len(self.target) and len(t) > self.last_len:
344             if t[-1] != self.target[len(t) - 1]: self.errs += 1
345 
346         self.last_len = len(t)
347 
348         html_parts = []
349         for i, char in enumerate(self.target):
350             disp = " " if char == " " else ("&lt;" if char == "<" else ("&gt;" if char == ">" else char))
351 
352             if i < len(t):
353                 if t[i] == char:
354                     html_parts.append(f'<span style="color: #10b981;">{disp}</span>')
355                 else:
356                     html_parts.append(f'<span style="color: #ef4444; background-color: #fee2e2;">{disp}</span>')
357             else:
358                 html_parts.append(f'<span style="color: #94a3b8;">{disp}</span>')
359 
360         scroll_pos = self.inp.horizontalScrollBar().value()
```

Рисунок 16 – Фрагмент исходного кода функции on_change

```python
362         full_html = f"<div style=\"white-space: nowrap;\">{''.join(html_parts)}</div>"
363         self.ref.setHtml(full_html)
364         self.ref.horizontalScrollBar().setValue(scroll_pos)
365 
366         if len(t) >= len(self.target):
367             self.finish_game()
```

Рисунок 17 – Фрагмент исходного кода функции on_change

Функция tick выполняет обновление таймера и расчёт текущей скорости печати. Исходный код функции представлен на рисунке 18.

```
369     def tick(self):  1 usage
370         if not self.start_t: return
371         elaps = time.time() - self.start_t
372         if self.time_limit > 0:
373             rem = max(0, int(self.time_limit - elaps))
374             m, sec = divmod(rem, 60)
375             self.lbl_timer.setText(f"{m:02d}:{sec:02d}")
376             if rem <= 0: self.finish_game(); return
377         cpm = int((len(self.inp.toPlainText()) / elaps) * 60) if elaps > 0 else 0
378         self.s_speed.val.setText(str(cpm))
```

Рисунок 18 – Исходный код функции tick

Функция finish_game завершает тренировку, вычисляет итоговую статистику и передаёт данные на экран статистики. Исходный код функции представлен на рисунке 19.

```
380     def finish_game(self):  2 usages
381         self.timer.stop()
382         self.active = False
383         self.inp.setReadOnly(True)
384         t = self.inp.toPlainText()
385         elaps = max(1, time.time() - self.start_t)
386         cpm = int((len(t) / elaps) * 60)
387         acc = max(0, int(((len(t) - self.errs) / max(len(t), 1)) * 100))
388         self.parent_app.stats_data = {"speed": cpm, "accuracy": acc, "errors": self.errs}
389         self.parent_app.stats_scr.update_stats()
390         self.parent_app.stack.setCurrentIndex(5)
```

Рисунок 19 – Исходный код функции finish_game

Класс StatsScreen отвечает за отображение результатов тренировки.

Функция make_row создаёт строку статистики. Исходный код функции представлен на рисунке 20.

```
441     def restart_game(self):  1 usage
442         self.parent_app.train_scr.init_game()
443         self.parent_app.stack.setCurrentIndex(1)
444
```

Рисунок 20 – Исходный код функции make_row

Функция update_stats обновляет данные о скорости, точности и количестве ошибок. Исходный код функции представлен на рисунке 21.

```
445     def make_row(self, title, color, bg):  3 usages
446         f = QFrame()
447         f.setStyleSheet(f"background-color: {bg}; border-radius: 11px;")
448         f.setFixedHeight(100)
449         l = QHBoxLayout(f)
450         icon = QFrame()
451         icon.setFixedSize(40, 40)
452         icon.setStyleSheet(f"background-color: {color}; border-radius: 5px;")
453         v_lay = QVBoxLayout()
454         t = QLabel(title)
455         t.setStyleSheet("color: #64748b; font-size: 15px; background: transparent;")
456         v = QLabel("0")
457         v.setStyleSheet(f"color: {color}; font-size: 25px; font-weight: bold; background: transparent;")
458         v_lay.addWidget(t)
459         v_lay.addWidget(v)
460         l.addWidget(icon)
461         l.addLayout(v_lay)
462         l.addStretch()
463         f.val_lbl = v
464         return f
```

Рисунок 21 – Исходный код функции update_stats

Функция restart_game запускает тренировку заново. Исходный код функции представлен на рисунке 22.

```
466     def update_stats(self):  1 usage (1 dynamic)
467         s = self.parent_app.stats_data
468         self.row_speed.val_lbl.setText(f"{s['speed']} зн/мин")
469         self.row_acc.val_lbl.setText(f"{s['accuracy']}%")
470         self.row_err.val_lbl.setText(str(s['errors']))
```

Рисунок 22 – Исходный код функции restart_game

Класс TrainerApp является главным классом приложения.

Функция __init__ создаёт главное окно программы, задаёт его размеры, применяет стили, создаёт все экраны приложения и добавляет их в QStackedWidget. Исходный код класса представлен на рисунке 23.

```
474 class TrainerApp(QMainWindow):  1 usage
475     def __init__(self):
476         super().__init__()
477         self.setWindowTitle("Тренажер")
478         self.resize(900, 850)
479         self.setStyleSheet(STYLE_SHEET)
480         self.settings = {"Раскладка": "Русская", "Режим": "С таймером", "Сложность": "Средний"}
481         self.stats_data = {"speed": 0, "accuracy": 0, "errors": 0}
482         self.stack = QStackedWidget()
483         self.setCentralWidget(self.stack)
484
485         self.menu_scr = MenuScreen(self)
486         self.train_scr = TrainingScreen(self)
487         self.sel_lang = SelectionScreen(self,  title: "Раскладка",  options: ["Русская", "Английская"],  setting_key: "Раскладка")
488         self.sel_mode = SelectionScreen(self,  title: "Режим",  options: ["С таймером", "Без таймера"],  setting_key: "Режим")
489         self.sel_diff = SelectionScreen(self,  title: "Сложность",  options: ["Лёгкий", "Средний", "Сложный"],  setting_key: "Сложность")
490         self.stats_scr = StatsScreen(self)
491
492         self.stack.addWidget(self.menu_scr)
493         self.stack.addWidget(self.train_scr)
494         self.stack.addWidget(self.sel_lang)
495         self.stack.addWidget(self.sel_mode)
496         self.stack.addWidget(self.sel_diff)
497         self.stack.addWidget(self.stats_scr)
```

Рисунок 23 – Исходный код класса TrainerApp

4.3.2 Разработка пользовательского интерфейса

Графический интерфейс приложения реализован на базе библиотеки PyQt6, которая предоставляет широкий набор готовых компонентов для создания современных настольных приложений. Интерфейс приложения выполнен в современном минималистичном стиле с использованием светлой цветовой гаммы. Для стилизации элементов используется технология QSS (Qt Style Sheets) – аналог CSS в Qt. Основные параметры оформления задаются в глобальной переменной (представлено на рис. 24):

```
11 STYLE_SHEET = """
12 QWidget { background-color: #F0F4F8; font-family: 'Segoe UI'; }
```

Рисунок 24 – Фрагмент исходного кода

С помощью таблицы стилей были реализованы:

- единый фон приложения;
- стилизованные карточки;
- кнопки навигации;
- кнопки выбора;

- кнопки закрытия;
- эффекты при наведении курсора.

Интерфейс приложения построен по принципу многоэкранной структуры. Для переключения между экранами используется виджет QStackedWidget, который позволяет хранить несколько экранных форм и отображать только одну активную форму. Создание контейнера экранов реализовано в главном классе приложения и показано на рисунке 25.

```
482        self.stack = QStackedWidget()
483        self.setCentralWidget(self.stack)
```

Рисунок 25 – Фрагмент исходного кода с использованием QStackedWidget

Добавление в контейнер всех основных экранов показано на рисунке 26.

```
492        self.stack.addWidget(self.menu_scr)
493        self.stack.addWidget(self.train_scr)
494        self.stack.addWidget(self.sel_lang)
495        self.stack.addWidget(self.sel_mode)
496        self.stack.addWidget(self.sel_diff)
497        self.stack.addWidget(self.stats_scr)
```

Рисунок 26 – Добавление всех основных экранов

Таким образом реализована удобная навигация между окнами без открытия новых форм.

Главное меню реализовано в классе MenuScreen. Данный экран содержит основные параметры тренировки и кнопку запуска. Каждый параметр представлен в виде кнопки-строки (представлено на рис. 27).

```
183        for key, idx in [("Раскладка", 2), ("Режим", 3), ("Сложность", 4)]:
184            btn = QPushButton()
185            btn.setProperty( name: "class",  value: "MenuRow")
186            btn.setFixedHeight(90)
187            blay = QVBoxLayout(btn)
188            t = QLabel(key)
189            t.setStyleSheet("color: #64748b; font-size: 20px; background: transparent;")
190            v = QLabel(parent.settings[key])
191            v.setStyleSheet("font-weight: bold; font-size: 24px; background: transparent;")
192            blay.addWidget(t)
193            blay.addWidget(v)
194            btn.clicked.connect(lambda ch, i=idx: self.parent_app.stack.setCurrentIndex(i))
195            card_lay.addWidget(btn)
196            self.labels[key] = v
```

Рисунок 27 – Фрагмент исходного кода с реализацией кнопок

Экран выбора реализован в классе SelectionScreen. Данный экран используется для выбора одного из параметров тренировки. Для отображения вариантов используются кнопки, выбранный вариант визуально выделяется. Для выбора параметра, пользователь должен нажать кнопку сохранить.

Экран тренировки реализован в классе TrainingScreen и является основным рабочим экраном приложения. Он содержит таймер, поле отображения эталонного текста, поле ввода текста, индикатор скорости, индикатор точности. Для отображения времени используется метка self.lbl_timer = QLabel("00:00"). Для отображения эталонного текста используется self.ref, а для ввода текста self.inp.

Фрагмент исходного кода для окраски символов представлен на рисунке 28.

```
352            if i < len(t):
353                if t[i] == char:
354                    html_parts.append(f'<span style="color: #10b981;">{disp}</span>')
355                else:
356                    html_parts.append(f'<span style="color: #ef4444; background-color: #fee2e2;">{disp}</span>')
357            else:
358                html_parts.append(f'<span style="color: #94a3b8;">{disp}</span>')
359
```

Рисунок 28 – Фрагмент исходного кода для визуализации правильности ввода

Для реализации таймера используется объект QTimer. Каждые 500 миллисекунд происходит обновление таймера и пересчёт скорости печати.

Экран статистики реализован в классе StatsScreen. После завершения тренировки на данном экране отображаются итоговые показатели.

Данные обновляются в функции update_stats. Исходный код функции показан на рисунке 29.

```
466     def update_stats(self):  1 usage (1 dynamic)
467         s = self.parent_app.stats_data
468         self.row_speed.val_lbl.setText(f"{s['speed']} зн/мин")
469         self.row_acc.val_lbl.setText(f"{s['accuracy']}%")
470         self.row_err.val_lbl.setText(str(s['errors']))
471
```

Рисунок 29 – Исходный код функции update_stats

Для построения интерфейса использованы следующие элементы библиотеки PyQt6: QWidget (базовый контейнер для экранов), QLabel (отображение текста), QPushButton (интерактивные кнопки), QTextEdit (поля отображения и ввода текста), QFrame (контейнеры и карточки), QVBoxLayout и QHBoxLayout (вертикальное и горизонтальное расположение элементов), QStackedWidget (переключение экранов), QTimer (реализация таймера), QGraphicsDropShadowEffect (создание эффекта тени), QTextCursor (управление положением курсора) и QColor (настройка цветовых параметров).

Вывод: в рамках данного раздела было разработано программное обеспечение «Клавиатурный тренжаер» на языке Python с использованием библиотеки PyQt6. Разработанная архитектура приложения включает шесть классов, отвечающих за пользовательский интерфейс, обработку настроек, тренировочный процесс и отображение статистики. Созданный графический интерфейс обеспечивает удобное взаимодействие пользователя с программой, позволяет настраивать параметры тренировки, контролировать скорость и точность печати, а также получать результаты после завершения сеанса.

Приложение А

(обязательное)

Исходный код

```
import sys
import random
import time
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QLabel, QPushButton, QStackedWidget,
                             QTextEdit, QGraphicsDropShadowEffect, QFrame)
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtGui import QColor, QTextCursor

# --- СТИЛИ (QSS) ---
STYLE_SHEET = """
QWidget { background-color: #F0F4F8; font-family: 'Segoe UI'; }
.Card { background-color: white; border-radius: 20px; }
.PrimaryButton {
    background-color: #3b82f6; color: white; border-radius: 16px;
    font-size: 16px; font-weight: bold; padding: 20px; border: none;
}
.PrimaryButton:hover { background-color: #2563eb; }
.MenuRow {
    background-color: #F8FAFC; border-radius: 16px;
    border: 3px solid #E2E8F0; text-align: left; p
}
.OptionButton {
    background-color: #F8FAFC; border-radius: 15px; border: 2px solid transparent;
    padding: 15px; text-align: left; font-size: 20px;
}
.OptionButton[selected="true"] { background-color: #EFF6FF; border: 6px solid #3b82f6; color:
#1d4ed8; }
.NavButton { background-color: white; border-radius: 15px; border: 3px solid #E2E8F0; font-
weight: bold; font-size: 25px; }
.CloseButton {
    background-color: white; border-radius: 15px; border: 3px solid #E2E8F0;
    font-weight: bold; font-size: 25px; color: #64748b;
}
.CloseButton:hover { background-color: #fee2e2; color: #ef4444; border: 3px solid #f87171; }
"""

VOCABULARY = {
    "Русская": {
        "Лёгкий": ["кот", "дом", "лес", "шар", "сон", "мир", "сок", "час", "друг", "мост",
"лето", "зима", "вода",
                   "небо"],
        "Средний": [
            "Быстрая лиса прыгает через ленивую собаку.",
            "Регулярная практика делает из новичка мастера.",
            "Каждый новый день приносит нам новые возможности.",
            "Чтение хороших книг сильно расширяет кругозор."
        ],
        "Сложный": [
            "В 2024 году вышло глобальное обновление версии 2.0!",
            "Синхрофазотрон: работает ли он на 100% мощности?",
            "Формула E=mc^2 была предложена А. Эйнштейном.",
            "Email для связи: test-email@domain.com (круглосуточно)."
        ]
    },
    "Английская": {
        "Лёгкий": ["cat", "dog", "sun", "fun", "car", "sky", "red", "big", "tree", "bird",
"blue", "wind", "star"],
        "Средний": [
            "The quick brown fox jumps over the lazy dog.",
            "Practice makes perfect in typing skills.",
            "Every journey begins with a single step.",
            "Reading books improves your vocabulary."
        ],
        "Сложный": [
            "In 2026, the version 3.0 update was released!",
            "Is the remote server running at 100% capacity?",
            "The formula E=mc^2 is quite famous (Einstein).",
            "Contact us at: support@domain.com (24/7)."
```

```
        ]
    }
}


def generate_practice_text(lang, diff, is_timer):
    pool = VOCABULARY[lang][diff]
    if diff == "Лёгкий":
        count = 30 if is_timer else 15
    else:
        count = 15 if is_timer else 4
    return " ".join(random.choices(pool, k=count))


class ShadowFrame(QFrame):
    def __init__(self):
        super().__init__()
        self.setProperty("class", "Card")
        shadow = QGraphicsDropShadowEffect()
        shadow.setBlurRadius(20)
        shadow.setColor(QColor(0, 0, 0, 20))
        shadow.setOffset(0, 5)
        self.setGraphicsEffect(shadow)


# --- ЭКРАН ВЫБОРА ---
class SelectionScreen(QWidget):
    def __init__(self, parent, title, options, setting_key):
        super().__init__(parent)
        self.parent_app = parent
        self.key, self.val = setting_key, parent.settings[setting_key]
        layout = QVBoxLayout(self)
        layout.setContentsMargins(30, 30, 30, 30)

        top_bar = QHBoxLayout()
        back = QPushButton("←")
        back.setFixedSize(45, 45)
        back.setProperty("class", "NavButton")
        back.clicked.connect(lambda: self.parent_app.stack.setCurrentIndex(0))
        close = QPushButton("×")
        close.setFixedSize(45, 45)
        close.setProperty("class", "CloseButton")
        close.clicked.connect(self.window().close)
        t_lbl = QLabel(title)
        t_lbl.setStyleSheet("font-size: 30px; font-weight: bold;")
        top_bar.addWidget(back)
        top_bar.addStretch()
        top_bar.addWidget(t_lbl)
        top_bar.addStretch()
        top_bar.addWidget(close)
        layout.addLayout(top_bar)

        layout.addStretch()
        container = QWidget()
        container.setFixedWidth(500)
        c_lay = QVBoxLayout(container)
        self.card = ShadowFrame()
        card_lay = QVBoxLayout(self.card)
        self.btns = []
        for o in options:
            btn = QPushButton(o)
            btn.setProperty("class", "OptionButton")
            btn.setProperty("selected", str(o == self.val).lower())
            btn.clicked.connect(lambda ch, v=o: self.select(v))
            card_lay.addWidget(btn)
            self.btns.append(btn)
        c_lay.addWidget(self.card)
        save = QPushButton("Сохранить")
        save.setProperty("class", "PrimaryButton")
        save.clicked.connect(self.save_data)
        c_lay.addWidget(save)
        layout.addWidget(container, alignment=Qt.AlignmentFlag.AlignCenter)
        layout.addStretch()

    def select(self, v):
        self.val = v
        for b in self.btns:
            b.setProperty("selected", str(b.text() == v).lower())
            b.style().unpolish(b)
            b.style().polish(b)
```

```python
    def save_data(self):
        self.parent_app.settings[self.key] = self.val
        self.parent_app.menu_scr.refresh_ui()
        self.parent_app.stack.setCurrentIndex(0)


# --- ЭКРАН МЕНЮ ---
class MenuScreen(QWidget):
    def __init__(self, parent):
        super().__init__(parent)
        self.parent_app = parent
        layout = QVBoxLayout(self)
        layout.setContentsMargins(30, 30, 30, 30)

        top_bar = QHBoxLayout()
        top_bar.addStretch()
        close = QPushButton("x")
        close.setFixedSize(45, 45)
        close.setProperty("class", "CloseButton")
        close.clicked.connect(self.window().close)
        top_bar.addWidget(close)
        layout.addLayout(top_bar)

        layout.addStretch()
        container = QWidget()
        container.setFixedWidth(500)
        c_lay = QVBoxLayout(container)
        title = QLabel("Меню")
        title.setStyleSheet("font-size: 30px; font-weight: bold;")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        c_lay.addWidget(title)
        c_lay.addSpacing(25)

        card = ShadowFrame()
        card_lay = QVBoxLayout(card)
        self.labels = {}
        for key, idx in [("Раскладка", 2), ("Режим", 3), ("Сложность", 4)]:
            btn = QPushButton()
            btn.setProperty("class", "MenuRow")
            btn.setFixedHeight(90)
            blay = QVBoxLayout(btn)
            t = QLabel(key)
            t.setStyleSheet("color: #64748b; font-size: 20px; background: transparent;")
            v = QLabel(parent.settings[key])
            v.setStyleSheet("font-weight: bold; font-size: 24px; background: transparent;")
            blay.addWidget(t)
            blay.addWidget(v)
            btn.clicked.connect(lambda ch, i=idx: self.parent_app.stack.setCurrentIndex(i))
            card_lay.addWidget(btn)
            self.labels[key] = v

        c_lay.addWidget(card)
        c_lay.addSpacing(25)
        start = QPushButton("Начать")
        start.setProperty("class", "PrimaryButton")
        start.clicked.connect(self.start_training)
        c_lay.addWidget(start)
        layout.addWidget(container, alignment=Qt.AlignmentFlag.AlignCenter)
        layout.addStretch()

    def refresh_ui(self):
        for k, v in self.parent_app.settings.items():
            if k in self.labels: self.labels[k].setText(v)

    def start_training(self):
        self.parent_app.train_scr.init_game()
        self.parent_app.stack.setCurrentIndex(1)


# --- ЭКРАН ТРЕНИРОВКИ ---
class TrainingScreen(QWidget):
    def __init__(self, parent):
        super().__init__(parent)
        self.parent_app = parent
        self.timer = QTimer()
        self.timer.timeout.connect(self.tick)
        self.last_len = 0

        layout = QVBoxLayout(self)
```

```
        layout.setContentsMargins(40, 40, 40, 40)

        top_bar = QHBoxLayout()
        back = QPushButton("←")
        back.setFixedSize(45, 45)
        back.setProperty("class", "NavButton")
        back.clicked.connect(self.go_back)
        close = QPushButton("×")
        close.setFixedSize(45, 45)
        close.setProperty("class", "CloseButton")
        close.clicked.connect(self.window().close)
        self.lbl_timer = QLabel("00:00")
        self.lbl_timer.setStyleSheet("font-size: 30px; font-weight: bold; color: #1e293b;")
        top_bar.addWidget(back)
        top_bar.addStretch()
        top_bar.addWidget(self.lbl_timer)
        top_bar.addStretch()
        top_bar.addWidget(close)
        layout.addLayout(top_bar)

        font_style = "font-family: 'Consolas', 'Courier New', monospace; font-size: 30px;
padding: 16px;"

        self.ref_container = ShadowFrame()
        self.ref_lay = QVBoxLayout(self.ref_container)
        self.ref_lay.setContentsMargins(0, 0, 0, 0)
        self.ref = QTextEdit()
        self.ref.setReadOnly(True)
        self.ref.setLineWrapMode(QTextEdit.LineWrapMode.NoWrap)
        self.ref.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        self.ref.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        self.ref.setStyleSheet(font_style + " color: #94a3b8; background: transparent; border:
none;")
        self.ref.setFixedHeight(80)
        self.ref_lay.addWidget(self.ref)
        layout.addWidget(self.ref_container)

        self.inp = QTextEdit()
        self.inp.setLineWrapMode(QTextEdit.LineWrapMode.NoWrap)
        self.inp.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        self.inp.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        self.inp.setStyleSheet(font_style + " border: 2px solid #3b82f6; border-radius: 12px;
background: white;")
        self.inp.setFixedHeight(70)
        self.inp.textChanged.connect(self.on_change)


self.inp.horizontalScrollBar().valueChanged.connect(self.ref.horizontalScrollBar().setValue)

        layout.addWidget(self.inp)
        layout.addSpacing(25)

        s_lay = QHBoxLayout()
        self.s_speed = self.stat_mini("Скорость", "0")
        self.s_acc = self.stat_mini("Точность", "100%")
        s_lay.addWidget(self.s_speed)
        s_lay.addWidget(self.s_acc)
        layout.addLayout(s_lay)
        layout.addStretch()

    def stat_mini(self, t, v):
        f = ShadowFrame()
        f.setFixedSize(200, 100)
        l = QVBoxLayout(f)
        l.setAlignment(Qt.AlignmentFlag.AlignCenter)
        lbl_t = QLabel(t)
        lbl_t.setStyleSheet("color: #64748b; font-size: 16px;")
        lbl_v = QLabel(v)
        lbl_v.setStyleSheet("font-size: 25px; font-weight: bold; color: #3b82f6;")
        l.addWidget(lbl_t)
        l.addWidget(lbl_v)
        f.val = lbl_v
        return f

    def init_game(self):
        s = self.parent_app.settings
        is_timer = s["Режим"] == "С таймером"
        self.target = generate_practice_text(s["Раскладка"], s["Сложность"], is_timer)

        self.errs, self.active, self.start_t = 0, False, None
```

```
        self.last_len = 0

        self.inp.clear()
        self.inp.setReadOnly(False)

        if is_timer:
            timer_mapping = {"Лёгкий": 15, "Средний": 30, "Сложный": 45}
            self.time_limit = timer_mapping[s["Сложность"]]
            m, sec = divmod(self.time_limit, 60)
            self.lbl_timer.setText(f"{m:02d}:{sec:02d}")
            self.lbl_timer.show()
        else:
            self.time_limit = 0
            self.lbl_timer.hide()

        self.on_change()

        self.s_speed.val.setText("0")
        self.s_acc.val.setText("100%")
        self.inp.setFocus()

    def go_back(self):
        self.timer.stop()
        self.parent_app.stack.setCurrentIndex(0)

    def on_change(self):
        t = self.inp.toPlainText()

        if '\n' in t:
            t = t.replace('\n', '')
            self.inp.setPlainText(t)
            cur = self.inp.textCursor()
            cur.movePosition(QTextCursor.MoveOperation.End)
            self.inp.setTextCursor(cur)
            return

        if not self.active and len(t) > 0:
            self.active = True
            self.start_t = time.time()
            self.timer.start(500)

        if len(t) > 0 and len(t) <= len(self.target) and len(t) > self.last_len:
            if t[-1] != self.target[len(t) - 1]: self.errs += 1

        self.last_len = len(t)

        html_parts = []
        for i, char in enumerate(self.target):
            disp = " " if char == " " else ("&lt;" if char == "<" else ("&gt;" if char ==
">" else char))

            if i < len(t):
                if t[i] == char:
                    html_parts.append(f'<span style="color: #10b981;">{disp}</span>')
                else:
                    html_parts.append(f'<span style="color: #ef4444; background-color:
#fee2e2;">{disp}</span>')
            else:
                html_parts.append(f'<span style="color: #94a3b8;">{disp}</span>')

        scroll_pos = self.inp.horizontalScrollBar().value()

        full_html = f"<div style=\"white-space: nowrap;\">{''.join(html_parts)}</div>"
        self.ref.setHtml(full_html)
        self.ref.horizontalScrollBar().setValue(scroll_pos)

        if len(t) >= len(self.target):
            self.finish_game()

    def tick(self):
        if not self.start_t: return
        elaps = time.time() - self.start_t
        if self.time_limit > 0:
            rem = max(0, int(self.time_limit - elaps))
            m, sec = divmod(rem, 60)
            self.lbl_timer.setText(f"{m:02d}:{sec:02d}")
            if rem <= 0: self.finish_game(); return
        cpm = int((len(self.inp.toPlainText()) / elaps) * 60) if elaps > 0 else 0
        self.s_speed.val.setText(str(cpm))
```

```
    def finish_game(self):
        self.timer.stop()
        self.active = False
        self.inp.setReadOnly(True)
        t = self.inp.toPlainText()
        elaps = max(1, time.time() - self.start_t)
        cpm = int((len(t) / elaps) * 60)
        acc = max(0, int(((len(t) - self.errs) / max(len(t), 1)) * 100))
        self.parent_app.stats_data = {"speed": cpm, "accuracy": acc, "errors": self.errs}
        self.parent_app.stats_scr.update_stats()
        self.parent_app.stack.setCurrentIndex(5)


# --- ЭКРАН СТАТИСТИКИ ---
class StatsScreen(QWidget):
    def __init__(self, parent):
        super().__init__(parent)
        self.parent_app = parent
        layout = QVBoxLayout(self)
        layout.setContentsMargins(30, 30, 30, 30)

        top_bar = QHBoxLayout()
        back = QPushButton("←")
        back.setFixedSize(45, 45)
        back.setProperty("class", "NavButton")
        back.clicked.connect(lambda: parent.stack.setCurrentIndex(0))
        close = QPushButton("×")
        close.setFixedSize(45, 45)
        close.setProperty("class", "CloseButton")
        close.clicked.connect(self.window().close)
        t_lbl = QLabel("Статистика")
        t_lbl.setStyleSheet("font-size: 30px; font-weight: bold;")
        top_bar.addWidget(back)
        top_bar.addStretch()
        top_bar.addWidget(t_lbl)
        top_bar.addStretch()
        top_bar.addWidget(close)
        layout.addLayout(top_bar)

        layout.addStretch()
        container = QWidget()
        container.setFixedWidth(500)
        c_lay = QVBoxLayout(container)
        self.card = ShadowFrame()
        card_lay = QVBoxLayout(self.card)
        self.row_speed = self.make_row("Скорость", "#3b82f6", "#EFF6FF")
        self.row_acc = self.make_row("Точность", "#10b981", "#ECFDF5")
        self.row_err = self.make_row("Ошибки", "#f97316", "#FFF7ED")
        card_lay.addWidget(self.row_speed)
        card_lay.addWidget(self.row_acc)
        card_lay.addWidget(self.row_err)
        c_lay.addWidget(self.card)
        c_lay.addSpacing(10)

        self.btn_restart = QPushButton("↻ Заново")
        self.btn_restart.setProperty("class", "PrimaryButton")
        self.btn_restart.clicked.connect(self.restart_game)
        c_lay.addWidget(self.btn_restart)
        layout.addWidget(container, alignment=Qt.AlignmentFlag.AlignCenter)
        layout.addStretch()

    def restart_game(self):
        self.parent_app.train_scr.init_game()
        self.parent_app.stack.setCurrentIndex(1)

    def make_row(self, title, color, bg):
        f = QFrame()
        f.setStyleSheet(f"background-color: {bg}; border-radius: 11px;")
        f.setFixedHeight(100)
        l = QHBoxLayout(f)
        icon = QFrame()
        icon.setFixedSize(40, 40)
        icon.setStyleSheet(f"background-color: {color}; border-radius: 5px;")
        v_lay = QVBoxLayout()
        t = QLabel(title)
        t.setStyleSheet("color: #64748b; font-size: 15px; background: transparent;")
        v = QLabel("0")
        v.setStyleSheet(f"color: {color}; font-size: 25px; font-weight: bold; background:
transparent;")
        v_lay.addWidget(t)
```

```
        v_lay.addWidget(v)
        l.addWidget(icon)
        l.addLayout(v_lay)
        l.addStretch()
        f.val_lbl = v
        return f

    def update_stats(self):
        s = self.parent_app.stats_data
        self.row_speed.val_lbl.setText(f"{s['speed']} зн/мин")
        self.row_acc.val_lbl.setText(f"{s['accuracy']}%")
        self.row_err.val_lbl.setText(str(s['errors']))


# --- ГЛАВНЫЙ КЛАСС ---
class TrainerApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Тренажер")
        self.resize(900, 850)
        self.setStyleSheet(STYLE_SHEET)
        self.settings = {"Раскладка": "Русская", "Режим": "С таймером", "Сложность": "Средний"}
        self.stats_data = {"speed": 0, "accuracy": 0, "errors": 0}
        self.stack = QStackedWidget()
        self.setCentralWidget(self.stack)

        self.menu_scr = MenuScreen(self)
        self.train_scr = TrainingScreen(self)
        self.sel_lang = SelectionScreen(self, "Раскладка", ["Русская", "Английская"],
"Раскладка")
        self.sel_mode = SelectionScreen(self, "Режим", ["С таймером", "Без таймера"], "Режим")
        self.sel_diff = SelectionScreen(self, "Сложность", ["Лёгкий", "Средний", "Сложный"],
"Сложность")
        self.stats_scr = StatsScreen(self)

        self.stack.addWidget(self.menu_scr)
        self.stack.addWidget(self.train_scr)
        self.stack.addWidget(self.sel_lang)
        self.stack.addWidget(self.sel_mode)
        self.stack.addWidget(self.sel_diff)
        self.stack.addWidget(self.stats_scr)


if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = TrainerApp()
    window.show()
    sys.exit(app.exec())
```